

AstraNotes — Week 6 Lab

Jonathan Fong | CSEN 296B | May 11, 2026

[Astra Notes GitHub Repository Link](#)

1. Development Direction

C++17 desktop application for macOS, Linux, and Windows. Local-first architecture with no network dependencies. Single-user, file-based persistence to ~/.astranotes/notes.json. Command-line interface (CLI) with simple menu navigation.

2. Project Structure and Build Environment

CMake 3.14+ build system with automatic dependency management via FetchContent. The src/ directory is organized by architectural layer: model/ (Note hierarchy and supporting types), controller/ (NoteFactory and NoteManager), storage/ (StorageInterface pure abstract), encryption/ (EncryptionEngine pure abstract), and view/ (CLIView boundary). Google Test v1.14.0 and nlohmann/json v3.11.3 are fetched at configure time; OpenSSL is declared but not linked this sprint. Both cmake --build and ctest exit 0 with no errors or warnings.

3. UI Shell Design

CLIView is a «boundary» class that serves as the thin presentation layer between the user and the controller. It exposes three methods per the UML: renderNoteList() displays all notes with index, UUID, type, and title; renderNote() shows full details (UUID, type, title, creation timestamp) for a selected note; promptInput() displays a prompt, reads a line from stdin, and returns trimmed input. The CLI menu offers four commands: [n] new note (creates a TextNote with user-supplied title), [l] list notes (calls renderNoteList), [s] settings (stub prints 'Settings: not yet implemented.'), and [q] quit. The settings stub satisfies the Week 6 lab requirement for a profile/settings menu entry while preserving design integrity (no SettingsManager or ProfileService class is created, as neither appears in the validated UML).

4. First Functionality Slices

Slice 1: Note Creation (US-01 / FR-1)

What it realizes:

Requirement: FR-1 (refined) — 'The system shall create a TextNote, VoiceNote, or SecureNote identified by a system-generated UUID (RFC 4122 v4) and a user-supplied title of 1–255 non-whitespace characters. Creation is performed exclusively through NoteFactory.'

UML Elements: NoteFactory (stateless factory), NoteManager.add() (orchestration), Note hierarchy (base class + TextNote subtype)

What was implemented:

NoteFactory is a stateless factory that allocates `std::unique_ptr<Note>` for TextNote and VoiceNote types. It generates RFC 4122 v4-style UUIDs using `thread_local std::mt19937_64` with proper version and variant bits set. It validates title length (rejects empty or whitespace-only strings) and throws `std::invalid_argument` on invalid input. `NoteManager.add()` transfers ownership of the note into its internal `std::map<UUID, std::unique_ptr<Note>>`. It asserts that no UUID collision occurs and throws `std::invalid_argument` if a duplicate is detected. `NoteManager.findByUUID()` performs a map lookup in $O(1)$ average time and returns a const pointer to the note or nullptr if not found. Both classes carry traceability comments citing FR-1 and the relevant UML elements.

Acceptance criteria covered:

Acceptance Criterion	Status
AC-01-1: Given the user enters a non-empty title, when [n] is selected in the CLI, then a TextNote with a UUID is created and appears in the note list.	✓ Implemented
AC-01-2: Given the user enters an empty string as the title, when NoteFactory.create is called, then a std::invalid_argument exception is thrown and no note is added to NoteManager.	✓ Implemented
AC-01-3: Given two notes are created with the same title, each shall receive a distinct UUID.	✓ Implemented
AC-01-4: Given I create a SecureNote, when the note is saved, then no plaintext content is stored in any data member or written to any file.	✗ Deferred (encryption not yet implemented)

5. Traceability Mapping

Requirement ID	Requirement Text	UML Element(s)	Status
FR-1	System shall create note with UUID and title	NoteFactory, NoteManager.add, Note hierarchy	✓
US-01 AC-1	System creates note with supplied title	NoteManager.add	✓
US-01 AC-2	Empty title rejected	NoteFactory.create (validation gate)	✓
US-01 AC-3	Two notes with same title receive distinct UUIDs	NoteFactory (UUID generation)	✓
NFR-1	UUID-keyed lookup in $O(1)$ time	NoteManager.findByUUID	✓

6. AI Reflection

How Claude Code Helped

Claude Code generated complete header and implementation scaffolding for all model, controller, view, and interface classes. The generated NoteFactory interface with method signatures for `create()` and `reconstructRecord()` was faster than hand-writing, as was the full Note abstract base class with derived TextNote, VoiceNote, and SecureNote implementations. CMakeLists.txt configuration with FetchContent for Google Test and nlohmann/json was generated correctly on the first pass. The RFC 4122 v4 UUID generation algorithm with proper bit manipulation was included without errors. Test case scaffolding with GTest syntax and test discovery was correct. Main.cpp CLI loop structure with menu parsing was functional immediately.

Quality Lesson Learned

When Claude Code asks for permission to create a file, reviewing the header documentation first is a reliable quality signal. Headers with comprehensive comment blocks on every method, ownership semantics clearly stated, and traceability comments already present indicate a high-quality first pass that requires minimal iteration. Conversely, headers with sparse documentation or vague ownership notes signal that the implementation will likely have matching gaps. Additionally, enforcing the Working Agreement's acceptance gates (explainability, charter alignment, memory safety, hallucination scan) before accepting any code caught ownership mistakes early—switching from `shared_ptr` to references would have been much more painful to fix after integration into dependent classes. This week reinforced that the agreement's gate structure, while time-consuming upfront, prevents costly rework downstream.